

Trabajo Práctico 2

SDN - NAT

García Lucentini Sabrina - 108751
Díaz Racioppi Valentina - 110998
Barberis Juan Celestino - 105147
Pérez Martinez Emanuel - 111855
Vallejo Tima Maximiliano - 112824
Valletta Franco - 111504

23 de Junio de 2026

1. Introducción.....	4
2. Hipótesis y soluciones.....	5
2.1. Soporte para múltiples hosts privados.....	5
2.2. Resolución dinámica de direcciones MAC.....	5
2.3. Traducción de direcciones y puertos.....	5
2.4. Mantenimiento del estado de las conexiones.....	6
2.5. Minimización de la intervención del controlador.....	6
3. Diseño de la tabla NAT.....	7
4. Manejo de ARP.....	9
5. Estrategia de instalación de flujos.....	10
6. Decisiones de diseño.....	11
6.1. Uso de una tabla NAT basada en la quintupla.....	11
6.2. Asignación dinámica de puertos públicos.....	11
6.3. Aprendizaje dinámico de direcciones MAC.....	11
6.4. Cola de paquetes pendientes durante la resolución ARP.....	11
6.5. Instalación temprana de reglas OpenFlow.....	11
6.6. Administración del ciclo de vida mediante FlowRemoved.....	12
6.7. Soporte limitado a IPv4, TCP y UDP.....	12
7. Problemas encontrados y soluciones implementadas.....	13
7.1. Resolución dinámica de direcciones MAC.....	13
7.2. Reenvío de paquetes durante la resolución ARP.....	13
7.3. Identificación correcta de conexiones simultáneas.....	13
7.4. Sincronización entre la tabla NAT y los flujos OpenFlow.....	13
7.5. Reescritura consistente de encabezados.....	14
7.6. Minimización de la carga sobre el controlador.....	14
8. Pruebas realizadas.....	15
8.1. Conectividad básica.....	16
8.2. Múltiples clientes.....	16
8.3. Diferentes tipos de tráfico.....	16
9. Conclusión.....	16

1. Introducción

El presente trabajo práctico tiene como objetivo implementar un sistema de Network Address Translation (NAT) con traducción de puertos (PAT) utilizando el paradigma de Redes Definidas por Software (SDN). Para ello se empleó Mininet como entorno de emulación de red, OpenFlow como protocolo de comunicación entre el plano de control y el plano de datos, y POX como controlador encargado de definir el comportamiento de la red.

A diferencia de una implementación tradicional de NAT realizada en dispositivos de red dedicados, en este trabajo la lógica de traducción y mantenimiento de estado se implementa en el controlador SDN. De esta manera, el controlador es responsable de procesar los primeros paquetes de cada flujo, resolver direcciones mediante ARP, crear las entradas de traducción necesarias e instalar reglas OpenFlow en el switch para que el tráfico posterior sea procesado directamente en el plano de datos.

La solución desarrollada permite que múltiples hosts pertenecientes a una red privada compartan una única dirección IP pública para acceder a la red externa. Para ello se implementó traducción de direcciones y puertos para tráfico TCP y UDP, una tabla NAT dinámica para el seguimiento de conexiones activas, mecanismos de resolución ARP dinámica y una estrategia de instalación de flujos orientada a minimizar la intervención del controlador una vez establecida cada conexión.

En las siguientes secciones se describe el diseño de la solución implementada, el funcionamiento de la tabla NAT, el manejo de ARP, la estrategia utilizada para la instalación de flujos OpenFlow, las decisiones de diseño adoptadas y las pruebas realizadas para validar el correcto funcionamiento del sistema.

2. Hipótesis y soluciones

Antes de comenzar la implementación se planteó la hipótesis de que era posible reproducir el comportamiento básico de un dispositivo NAT/PAT utilizando un controlador SDN, delegando en el switch OpenFlow únicamente las tareas de reenvío y modificación de paquetes una vez establecidas las reglas correspondientes.

Para cumplir con los requisitos del trabajo se identificaron los siguientes problemas a resolver y las soluciones correspondientes a cada una de ellas.

2.1. Soporte para múltiples hosts privados

Dado que la topología original sólo contemplaba un host privado, se planteó la necesidad de extender la red para validar el funcionamiento del NAT con múltiples clientes simultáneos.

Como solución se agregaron nuevos hosts dentro de la misma subred privada, permitiendo generar conexiones concurrentes hacia la red pública y verificar el correcto funcionamiento de la traducción de direcciones y puertos.

2.2. Resolución dinámica de direcciones MAC

La eliminación de las entradas ARP estáticas implicó que el controlador debía ser capaz de descubrir dinámicamente las direcciones físicas de los hosts conectados a la red.

Se propuso implementar una tabla ARP dinámica mantenida por el controlador, alimentada a partir de los paquetes observados en la red. Además, el controlador debería responder consultas ARP dirigidas a las interfaces virtuales del NAT y generar solicitudes ARP cuando necesita resolver la dirección MAC de un destino desconocido.

2.3. Traducción de direcciones y puertos

Para permitir que múltiples hosts privados compartieran una única dirección IP pública se planteó la implementación de un mecanismo de Port Address Translation (PAT).

La solución adoptada consiste en asignar dinámicamente un puerto público disponible a cada nueva conexión identificada por la combinación de dirección IP origen, puerto origen, dirección IP destino, puerto destino y protocolo de transporte. De esta forma es posible distinguir conexiones simultáneas aun cuando todas utilicen la misma dirección IP pública.

2.4. Mantenimiento del estado de las conexiones

Debido a que la traducción debe aplicarse tanto al tráfico saliente como al tráfico de respuesta, se consideró necesario mantener información de estado asociada a cada flujo activo.

Para ello se propuso utilizar una tabla NAT que almacene la relación entre cada conexión privada y el puerto público asignado, permitiendo realizar posteriormente la traducción inversa de los paquetes entrantes.

2.5. Minimización de la intervención del controlador

Uno de los objetivos principales de SDN es reducir el procesamiento realizado por el controlador una vez que el comportamiento de la red ha sido definido.

Por este motivo se propuso que el controlador intervenga únicamente ante nuevos flujos o eventos de resolución ARP. Una vez establecida una traducción NAT, se instalarán reglas OpenFlow en el switch para realizar automáticamente las modificaciones necesarias sobre los paquetes posteriores, disminuyendo la cantidad de mensajes PacketIn y mejorando la eficiencia del sistema.

3. Diseño de la tabla NAT

La funcionalidad principal del sistema desarrollado consiste en implementar un mecanismo de Network Address Translation con traducción de puertos (PAT), permitiendo que múltiples hosts de la red privada compartan una única dirección IP pública para comunicarse con la red externa.

Para identificar de forma unívoca cada conexión se definió una clave compuesta por cinco campos:

- Dirección IP origen privada.
- Puerto origen privado.
- Dirección IP destino.
- Puerto destino.
- Protocolo de transporte (TCP o UDP).

Esta información se encapsula mediante la estructura NatKey, que representa un flujo de comunicación individual. La utilización de la quintupla permite diferenciar conexiones simultáneas incluso cuando provienen del mismo host privado o se dirigen al mismo servidor.

A partir de esta clave, el controlador mantiene una tabla NAT que asocia cada conexión con un puerto público asignado dinámicamente. Cuando un host privado inicia una nueva comunicación, el controlador verifica si ya existe una traducción para dicha conexión. En caso contrario, selecciona un puerto libre dentro de un rango predefinido y registra la asociación correspondiente.

Conceptualmente, la tabla NAT mantiene entradas similares a las siguientes:

Protocolo	IP Privada	Puerto Privado	IP Destino	Puerto Destino	Puerto Publico
TCP	192.168.1.2	5000	200.0.0.1	80	10000
TCP	192.168.1.3	5000	200.0.0.1	80	10001
UDP	192.168.1.4	6000	200.0.0.1	53	10002

Tabla 1: Representación simplificada de las traducciones almacenadas por el controlador.

De esta manera, diferentes hosts privados pueden utilizar simultáneamente los mismos puertos locales sin generar conflictos, ya que cada conexión recibe un puerto público distinto.

Adicionalmente, se mantiene una estructura inversa que relaciona cada puerto público con su correspondiente entrada NAT. Esto permite localizar rápidamente la conexión original cuando se recibe tráfico proveniente de la red pública y realizar la traducción inversa hacia el host privado correcto.

La asignación de puertos públicos se realiza utilizando un conjunto de puertos disponibles. Cuando una nueva conexión es creada, se reserva uno de estos puertos y se lo marca como ocupado. Posteriormente, cuando la conexión expira y los flujos OpenFlow asociados son eliminados, el puerto es liberado y vuelve a estar disponible para futuras asignaciones.

Este diseño permite soportar múltiples conexiones concurrentes, evitar colisiones entre traducciones y reutilizar eficientemente los recursos disponibles, cumpliendo con los requisitos de un NAT por puertos implementado sobre una arquitectura SDN.

4. Manejo de ARP

Con el objetivo de cumplir con el requisito de eliminar cualquier configuración ARP estática, se implementó un mecanismo completo de resolución dinámica de direcciones MAC administrado por el controlador SDN.

Para ello se incorporó una tabla ARP dinámica que almacena asociaciones entre direcciones IP, direcciones MAC y puertos del switch. Esta información se obtiene mediante aprendizaje automático a partir de los paquetes que atraviesan la red. Cada vez que el controlador recibe un paquete ARP o IPv4, registra la dirección IP de origen, la dirección MAC correspondiente y el puerto por el cual fue recibido.

Cuando un host envía una solicitud ARP dirigida a alguna de las interfaces virtuales del NAT, el controlador genera y envía una respuesta ARP indicando la dirección MAC correspondiente a dicha interfaz. De esta manera, los hosts pueden resolver correctamente las direcciones del gateway sin necesidad de configuraciones manuales.

Por otro lado, cuando el controlador necesita reenviar un paquete hacia un destino cuya dirección MAC aún no conoce, consulta la tabla ARP. Si no existe una entrada para la dirección IP buscada, el paquete es almacenado temporalmente en una cola de pendientes y se genera automáticamente una solicitud ARP para resolver dicha dirección.

Una vez recibida la respuesta ARP correspondiente, la información es incorporada a la tabla ARP y todos los paquetes que se encontraban esperando dicha resolución son reprocesados. Este mecanismo evita la pérdida de tráfico durante el proceso de descubrimiento de direcciones.

Adicionalmente, se implementó un sistema de reintentos para las solicitudes ARP pendientes. Si una dirección no responde luego de una cantidad configurable de intentos, los paquetes asociados son descartados y la resolución se considera fallida. Esto evita que se acumulen indefinidamente paquetes en memoria ante hosts inaccesibles o inexistentes.

Gracias a este mecanismo, la solución desarrollada no depende de direcciones MAC preconfiguradas y puede adaptarse dinámicamente a cambios en la topología o en los dispositivos conectados, cumpliendo con los requisitos establecidos por el trabajo práctico.

5. Estrategia de instalación de flujos

Con el fin de aprovechar las ventajas del paradigma SDN y minimizar la carga de procesamiento sobre el controlador, se adoptó una estrategia basada en la instalación dinámica de reglas OpenFlow para cada nueva conexión detectada.

Cuando un host privado inicia una comunicación hacia la red pública, el primer paquete del flujo es enviado al controlador mediante un mensaje PacketIn. En ese momento, el controlador realiza todas las tareas necesarias: resolución ARP si corresponde, creación o búsqueda de la entrada NAT, asignación de un puerto público y traducción inicial del paquete.

Una vez completado este procesamiento, el controlador instala en el switch una regla de flujo para el tráfico saliente. Esta regla identifica los paquetes pertenecientes a la conexión mediante la combinación de dirección IP origen, dirección IP destino, puertos de transporte y protocolo. Las acciones asociadas a la regla realizan automáticamente la traducción de la dirección IP origen, la traducción del puerto origen y la reescritura de las direcciones MAC necesarias para el reenvío hacia la red pública.

De manera similar, cuando llega el primer paquete de respuesta desde la red pública, el controlador identifica la traducción NAT correspondiente y genera una segunda regla para el tráfico entrante. Esta regla realiza la traducción inversa, reemplazando la dirección IP pública y el puerto público por la dirección y el puerto originales del host privado, además de actualizar las direcciones MAC para su correcta entrega.

Como resultado, únicamente los primeros paquetes de cada conexión son procesados por el controlador. Una vez instaladas las reglas, el resto del tráfico es manejado directamente por el switch OpenFlow, evitando la generación de nuevos mensajes PacketIn para la misma comunicación.

Para liberar recursos automáticamente, los flujos se instalan con tiempos de expiración (idle timeout) dependientes del protocolo utilizado. Las conexiones TCP reciben un tiempo de vida mayor debido a su naturaleza orientada a conexión, mientras que las conexiones UDP utilizan un tiempo de expiración más corto. Cuando un flujo saliente expira, el switch notifica al controlador mediante un evento FlowRemoved. En respuesta, el controlador elimina el flujo entrante asociado, libera el puerto público asignado y elimina la entrada correspondiente de la tabla NAT.

Esta estrategia permite reducir significativamente la intervención del controlador durante el funcionamiento normal de la red, manteniendo únicamente las tareas de establecimiento de nuevas conexiones, mantenimiento del estado NAT y resolución ARP.

6. Decisiones de diseño

Durante el desarrollo de la solución se tomaron diversas decisiones de diseño con el objetivo de simplificar la implementación, cumplir con los requisitos del trabajo práctico y mantener una arquitectura consistente con los principios de SDN.

6.1. Uso de una tabla NAT basada en la quintupla

Para identificar de manera unívoca cada conexión se optó por utilizar como clave la combinación de dirección IP origen, puerto origen, dirección IP destino, puerto destino y protocolo de transporte. Esta decisión permite distinguir correctamente múltiples conexiones simultáneas, incluso cuando distintos hosts utilizan los mismos puertos locales o se comunican con el mismo servidor.

6.2. Asignación dinámica de puertos públicos

Se implementó un mecanismo de asignación dinámica de puertos públicos dentro de un rango predefinido. Cada nueva conexión recibe un puerto disponible que permanece reservado mientras la conexión se encuentra activa. Esta estrategia evita colisiones entre traducciones y permite reutilizar recursos una vez que las conexiones finalizan.

6.3. Aprendizaje dinámico de direcciones MAC

Se decidió eliminar completamente cualquier dependencia de configuraciones ARP estáticas. En su lugar, el controlador aprende dinámicamente las asociaciones entre direcciones IP, direcciones MAC y puertos del switch a partir del tráfico observado. Esto permite que la solución sea independiente de direcciones hardcodeadas y se adapte automáticamente a modificaciones en la topología.

6.4. Cola de paquetes pendientes durante la resolución ARP

Cuando el controlador necesita reenviar un paquete hacia un destino cuya dirección MAC aún no conoce, el paquete es almacenado temporalmente hasta completar la resolución ARP. Esta decisión evita descartar tráfico válido mientras se obtiene la información necesaria para el reenvío y mejora la robustez general del sistema.

6.5. Instalación temprana de reglas OpenFlow

Se optó por procesar únicamente los primeros paquetes de cada flujo en el controlador. Una vez creada la traducción NAT correspondiente, se instalan reglas OpenFlow tanto para el tráfico saliente como para el entrante. De esta manera, el switch puede realizar las

modificaciones necesarias directamente en el plano de datos, reduciendo la cantidad de eventos PacketIn y mejorando el rendimiento de la red.

6.6. Administración del ciclo de vida mediante FlowRemoved

Para evitar que las tablas NAT crezcan indefinidamente, se decidió utilizar los eventos FlowRemoved generados por OpenFlow cuando un flujo expira. Esta estrategia permite sincronizar automáticamente el estado almacenado por el controlador con las reglas instaladas en el switch, liberando puertos públicos y eliminando traducciones que ya no se encuentran en uso.

6.7. Soporte limitado a IPv4, TCP y UDP

Debido a los objetivos del trabajo práctico, la implementación se enfocó exclusivamente en tráfico IPv4 y protocolos de transporte TCP y UDP. Los demás protocolos son ignorados por el mecanismo NAT. Esta decisión simplifica la implementación y permite concentrar el desarrollo en los requisitos solicitados por la consigna.

7. Problemas encontrados y soluciones implementadas

Durante la implementación del controlador se presentaron diversos desafíos relacionados con el manejo de estado, la resolución de direcciones y la interacción entre el controlador SDN y el switch OpenFlow.

7.1. Resolución dinámica de direcciones MAC

Uno de los primeros inconvenientes surgió al eliminar las entradas ARP estáticas presentes en la topología inicial. Sin una resolución dinámica de direcciones MAC, el controlador no podía determinar correctamente el destino de los paquetes que debían ser reenviados.

Para resolver este problema se implementó una tabla ARP dinámica junto con mecanismos de generación y procesamiento de mensajes ARP Request y ARP Reply. Además, se incorporó una cola de paquetes pendientes para almacenar temporalmente los paquetes cuyo destino aún no había sido resuelto.

7.2. Reenvío de paquetes durante la resolución ARP

Inicialmente, los paquetes que requerían una dirección MAC desconocida eran descartados hasta obtener una respuesta ARP válida. Esto provocaba pérdidas de tráfico y fallas en el establecimiento de nuevas conexiones.

La solución consistió en almacenar dichos paquetes en una estructura de espera y volver a procesarlos automáticamente una vez completada la resolución ARP correspondiente.

7.3. Identificación correcta de conexiones simultáneas

Otro desafío fue garantizar que múltiples hosts privados pudieran comunicarse simultáneamente con la red pública sin generar conflictos en las traducciones.

La utilización exclusiva de direcciones IP resultaba insuficiente para diferenciar todas las conexiones activas. Por este motivo se adoptó una identificación basada en la quintupla formada por dirección IP origen, puerto origen, dirección IP destino, puerto destino y protocolo, permitiendo distinguir correctamente cada flujo de comunicación.

7.4. Sincronización entre la tabla NAT y los flujos OpenFlow

Durante las primeras pruebas se observó que las entradas NAT podían permanecer almacenadas aun cuando la comunicación ya había finalizado, provocando un uso innecesario de recursos y una reducción gradual de los puertos disponibles.

Para solucionar este problema se aprovecharon los eventos FlowRemoved generados por OpenFlow al expirar un flujo. De esta manera, cuando una regla deja de estar activa, el

controlador elimina automáticamente la traducción asociada y libera el puerto público correspondiente.

7.5. Reescritura consistente de encabezados

La traducción NAT requiere modificar simultáneamente información de distintas capas de la pila de protocolos. En particular, fue necesario reescribir direcciones IP, puertos TCP/UDP y direcciones MAC tanto en el tráfico saliente como en el tráfico entrante.

Las primeras versiones de la implementación presentaban inconsistencias entre algunas de estas modificaciones, lo que impedía que las respuestas regresaran correctamente al host privado correspondiente. Este problema se resolvió definiendo reglas de traducción simétricas para ambos sentidos de la comunicación e instalando flujos específicos para cada dirección del tráfico.

7.6. Minimización de la carga sobre el controlador

Finalmente, uno de los principales desafíos fue evitar que todos los paquetes fueran procesados por el controlador. Si bien esta estrategia simplifica la implementación inicial, genera una dependencia excesiva del plano de control y reduce la eficiencia general del sistema.

La solución adoptada consistió en utilizar el controlador únicamente para el establecimiento de nuevas conexiones y para eventos de resolución ARP. Una vez creada la traducción NAT correspondiente, las reglas OpenFlow instaladas permiten que el switch procese directamente el resto del tráfico asociado al flujo.

8. Pruebas realizadas

Para validar el correcto funcionamiento del sistema NAT/PAT basado en SDN, se diseñó un plan de pruebas dividido en tres etapas funcionales. Estas pruebas permiten comprobar desde la conectividad inicial y el comportamiento del plano de datos, hasta la capacidad de soportar múltiples clientes en simultáneo y la correcta reescritura de los encabezados en la red pública.

8.1. Conectividad básica

El objetivo es verificar que un host de la red privada pueda establecer una conexión TCP hacia la red pública de forma exitosa, comprobando el correcto funcionamiento de la resolución ARP dinámica, la creación de la primera entrada en la tabla NAT y la posterior instalación de flujos en el switch OpenFlow.

Se levantó un servidor TCP utilizando la herramienta iperf en el puerto 5001 dentro del host público h1 (200.0.0.1). Posteriormente, desde el host privado h2 (192.168.1.2), se inició una conexión cliente hacia dicha dirección IP pública.

Resultados y Análisis: Al registrarse el primer paquete del flujo, el controlador POX ejecutó de manera secuencial las tareas lógicas planificadas:

1. Resolución ARP: El controlador aprendió dinámicamente la MAC de h2 y, al no conocer la dirección física de h1, generó un ARP Request hacia la red pública, pausando el paquete en una cola de pendientes hasta recibir el ARP Reply.
2. Creación de Entrada NAT: Se generó la estructura NatKey asociando la quintupla del flujo privado a un puerto público libre (10240) asignado a la interfaz externa (200.0.0.254).
3. Instalación de flujos: Se inyectaron reglas específicas en el switch tanto para el camino de ida como para el de vuelta.

```
INFO:core:POX 0.7.0 (gar) is up.
DEBUG:openflow.of_01:Listening on 0.0.0.0:6633
INFO:openflow.of_01:[00:00:00:00:00:01 2] connected
INFO:protorouter:Iniciando ProtoRouter para Switch 1
DEBUG:openflow.of_01:1 connection aborted
INFO:protorouter:ARP REQUEST: ¿quién tiene 192.168.1.254? (preguntado por 192.168.1.2/00:00:00:00:00:02, puerto 2)
INFO:protorouter:ARP LEARN: 192.168.1.2 -> 00:00:00:00:00:02 (puerto 2)
INFO:protorouter:ARP REPLY: 192.168.1.254 está en 00:00:00:bb:bb:bb -> puerto 2
INFO:protorouter:RECIBIDO: 192.168.1.2 -> 200.0.0.1 | MAC: 00:00:00:00:00:02 -> 00:00:00:bb:bb:bb | In Port: 2
INFO:protorouter:MATCH: 192.168.1.2 pertenece a la red privada 192.168.1.0/24
INFO:protorouter:Paquete outgoing: 192.168.1.2:37690 -> 200.0.0.1:5001 (proto 6)
INFO:protorouter:MAC de 200.0.0.1 desconocida, resolviendo por ARP...
INFO:protorouter:ARP REQUEST enviado: ¿quién tiene 200.0.0.1? (desde 200.0.0.254/00:00:00:aa:aa:aa)
INFO:protorouter:ARP REPLY: 200.0.0.1 está en 00:00:00:00:00:01 (puerto 1)
INFO:protorouter:ARP LEARN: 200.0.0.1 -> 00:00:00:00:00:01 (puerto 1)
INFO:protorouter:ARP resuelto para 200.0.0.1: reprocesando 1 paquete(s) pendiente(s)
INFO:protorouter:RECIBIDO: 192.168.1.2 -> 200.0.0.1 | MAC: 00:00:00:00:00:02 -> 00:00:00:bb:bb:bb | In Port: 2
INFO:protorouter:MATCH: 192.168.1.2 pertenece a la red privada 192.168.1.0/24
INFO:protorouter:Paquete outgoing: 192.168.1.2:37690 -> 200.0.0.1:5001 (proto 6)
INFO:protorouter:Nueva entrada NAT creada: NatKey(src_ip=IPAddr('192.168.1.2'), src_port=37690, dst_ip=IPAddr('200.0.0.1'), dst_port=5001, proto=6) -> puerto público 10240
INFO:protorouter:ENVIANDO: 200.0.0.254 -> 200.0.0.1 | MAC: 00:00:00:aa:aa:aa -> 00:00:00:00:00:01 | Out Port: 5001
INFO:protorouter:Paquete traducido y enviado: 200.0.0.254:10240 -> 200.0.0.1:5001 (proto 6)
INFO:protorouter:FLUJO SALIENTE instalado: 192.168.1.2:37690 -> 200.0.0.1:5001 (pub port 10240)
INFO:protorouter:RECIBIDO: 200.0.0.1 -> 200.0.0.254 | MAC: 00:00:00:00:00:01 -> 00:00:00:aa:aa:aa | In Port: 1
INFO:protorouter:MATCH: 200.0.0.1 pertenece a la red pública
INFO:protorouter:INCOMING: 200.0.0.1:5001 -> 200.0.0.254:10240 (proto=6)
INFO:protorouter:NAT ENCONTRADO: puerto 10240 -> 192.168.1.2:37690
INFO:protorouter:PAQUETE DESTRADUCIDO: 200.0.0.1:5001 -> 192.168.1.2:37690
INFO:protorouter:FLUJO ENTRANTE instalado: 200.0.0.1:5001 -> pub port 10240 -> 192.168.1.2:37690
INFO:protorouter:ARP REQUEST: ¿quién tiene 200.0.0.254? (preguntado por 200.0.0.1/00:00:00:00:00:01, puerto 1)
INFO:protorouter:ARP REPLY: 200.0.0.254 está en 00:00:00:aa:aa:aa -> puerto 1
```

Figura 1: Log del controlador POX detallando el aprendizaje ARP, la resolución de la interfaz pública, la creación de la entrada NAT y la instalación de flujos salientes/entrantes.

Una vez instaladas las reglas en el switch, se verificó mediante el comando `ovs-ofctl dump-flows s1` que el resto del tráfico de la sesión de `iperf` se delegó por completo al plano de datos. El switch OpenFlow realizó la modificación de los encabezados de forma autónoma, alcanzando anchos de banda óptimos de transferencia (aprox. 41.4 Gbits/sec), lo que demuestra que el controlador no sufrió degradación por procesar paquetes subsiguientes.

The image shows two terminal windows. The top window, titled 'Node h1', displays the output of `ovs-ofctl dump-flows s1`, showing two flow entries for `s1-eth1` and `s1-eth2` with actions for modifying IP, MAC, and port. The bottom window, titled 'Node h2', shows the output of `iperf` running on port 5001, indicating a successful connection to 200.0.0.1 port 5001 with a transfer rate of 41.4 Gbits/sec.

Figura 2: Transferencia exitosa con `iperf` entre `h2` y `h1`, junto con el volcado de flujos de OpenFlow que evidencia las acciones de reescritura IP/MAC/Puerto.

8.2. Múltiples clientes

El objetivo es validar la capacidad del mecanismo PAT para diferenciar flujos concurrentes provenientes de distintos hosts privados que se dirigen al mismo destino y puerto público, utilizando una única dirección IP externa.

Manteniendo el servidor `iperf` activo en `h1`, se ejecutaron comandos de conexión en forma simultánea desde dos clientes privados distintos: `h2` (192.168.1.2) y `h3` (192.168.1.3), forzando una transferencia de volumen limitado (`-n 1KB`) para observar el comportamiento dinámico en los logs.

Resultados y Análisis: El controlador resolvió eficazmente la concurrencia aplicando la lógica basada en la quintupla. Como se observa en los registros, cada host privado recibió un puerto público único y consecutivo, garantizando el aislamiento de los canales de comunicación:

- Flujo `h2`: 192.168.1.2:50324 → Mapeado al puerto público 10240.
- Flujo `h3`: 192.168.1.3:40108 → Mapeado al puerto público 10241.

Ambas conexiones coexistieron de manera transparente, confirmando que la asignación dinámica de puertos evita colisiones y permite el escalamiento de hosts dentro de la LAN privada.

Figura 3: Conexión simultánea de los hosts h2 y h3. Los logs de POX confirman la creación de dos entradas NAT independientes (puertos públicos 10240 y 10241).

8.3. Diferentes tipos de tráfico

El objetivo es analizar a nivel de bits y paquetes la consistencia en la traducción de los encabezados de las capas de Red y Transporte (tráfico TCP e intercambios ARP de control) en la interfaz externa.

Se realizó una captura de tráfico mediante la herramienta Wireshark en la interfaz del host público (h1-eth0) durante el establecimiento de una sesión de comunicación, analizando el three-way handshake de TCP.

Resultados y Análisis: La inspección de paquetes confirmó de forma empírica el cumplimiento de las hipótesis de diseño:

- Antes del envío de datos, se observa el intercambio ARP local en el segmento público (Who has 200.0.0.1? Tell 200.0.0.254).
- En los paquetes TCP subsiguientes (segmentos SYN, SYN-ACK, ACK), se corroboró que la dirección IP de origen visible para el servidor externo es siempre la IP pública del NAT (200.0.0.254) y que los puertos modificados coinciden exactamente con los asignados en la tabla de estados del controlador (ej. puerto origen 10240).

Esto certifica que la reescritura es simétrica, consistente y totalmente invisible para los extremos de la comunicación.

Capturing from h1-eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter... <Ctrl>F

No.	Time	Source	Destination	Protocol	Length	Info
1	0.00000000	00:00:00:aa:aa:aa	Broadcast	ARP	42	who has 200.0.0.1? Tell 200.0.0.254
2	0.00034254	00:00:00:00:00:01	00:00:00:aa:aa:aa	ARP	42	200.0.0.1 is at 00:00:00:00:00:01
3	0.00248218	200.0.0.254	200.0.0.1	TCP	74	200.0.0.1 → 200.0.0.254 [RST] Seq=1 Win=0 Len=0 MSS=1460 SACK_PERM TSval=766852167 TSecr=766852167 WS=512
4	0.00306470	200.0.0.1	200.0.0.254	TCP	74	200.0.0.1 → 200.0.0.254 [SYN] ACK Seq=0 Ack=1 Win=4348 Len=0 MSS=1460 SACK_PERM TSval=1920273432 TSecr=766852167 WS=512
5	0.04520582	200.0.0.254	200.0.0.1	TCP	66	200.0.0.254 → 200.0.0.1 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=766852817 TSecr=1920273432
6	0.04535381	200.0.0.254	200.0.0.1	TCP	130	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=766852817 TSecr=1920273432
7	0.04536595	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=1 Ack=61 Win=43520 Len=0 TSval=1920273474 TSecr=766852817
8	0.045404156	200.0.0.254	200.0.0.1	TCP	2962	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=61 Ack=1 Win=42496 Len=2896 TSval=766852817 TSecr=1920273432
9	0.045410287	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=1 Ack=2957 Win=49664 Len=0 TSval=1920273474 TSecr=766852817
10	0.045423812	200.0.0.254	200.0.0.1	TCP	2962	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=2957 Ack=1 Win=42496 Len=2896 TSval=766852817 TSecr=1920273432
11	0.045429586	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=1 Ack=5853 Win=55296 Len=0 TSval=1920273474 TSecr=766852817
12	0.045440988	200.0.0.254	200.0.0.1	TCP	2962	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=5853 Ack=1 Win=42496 Len=2896 TSval=766852817 TSecr=1920273432
13	0.045445230	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=1 Ack=8749 Win=55296 Len=0 TSval=1920273474 TSecr=766852817
14	0.045455156	200.0.0.254	200.0.0.1	TCP	2962	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=8749 Ack=1 Win=42496 Len=2896 TSval=766852817 TSecr=1920273432
15	0.045458941	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=1 Ack=11645 Win=52768 Len=0 TSval=1920273474 TSecr=766852817
16	0.045460470	200.0.0.254	200.0.0.1	TCP	1514	200.0.0.254 → 200.0.0.1 [ACK] Seq=11645 Ack=1 Win=42496 Len=1448 TSval=766852817 TSecr=1920273432
17	0.045472223	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=1 Ack=13093 Win=52768 Len=0 TSval=1920273474 TSecr=766852817
18	0.045480907	200.0.0.1	200.0.0.254	TCP	84	200.0.0.1 → 200.0.0.254 [ACK] Seq=1 Ack=13093 Win=52768 Len=0 TSval=1920273474 TSecr=766852817
19	0.045614057	200.0.0.254	200.0.0.1	TCP	14548	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=13093 Ack=29 Win=42496 Len=14480 TSval=766852817 TSecr=1920273474
20	0.045622099	200.0.0.254	200.0.0.1	TCP	7390	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=22573 Ack=29 Win=42496 Len=7240 TSval=766852817 TSecr=1920273474
21	0.045635129	200.0.0.254	200.0.0.1	TCP	66	200.0.0.254 → 200.0.0.1 [ACK] Seq=22573 Ack=29 Win=42496 Len=0 TSval=766852817 TSecr=1920273474
22	0.045642159	200.0.0.254	200.0.0.1	TCP	7390	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=34813 Ack=29 Win=42496 Len=7240 TSval=766852818 TSecr=1920273474
23	0.045630788	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=42053 Win=55296 Len=0 TSval=1920273475 TSecr=766852817
24	0.045879993	200.0.0.254	200.0.0.1	TCP	14548	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=42053 Ack=29 Win=42496 Len=14480 TSval=766852818 TSecr=1920273475
25	0.045880051	200.0.0.254	200.0.0.1	TCP	17366	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=50533 Ack=29 Win=42496 Len=21720 TSval=766852818 TSecr=1920273475
26	0.045891852	200.0.0.254	200.0.0.1	TCP	20180	200.0.0.254 → 200.0.0.1 [TCP Window Full] Seq=12240 Len=0 MSS=1460 SACK_PERM TSval=766852818 TSecr=1920273475
27	0.045919128	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=98373 Win=55296 Len=0 TSval=1920273475 TSecr=766852818
28	0.045957605	200.0.0.254	200.0.0.1	TCP	6010	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=98373 Ack=29 Win=42496 Len=5844 TSval=766852818 TSecr=1920273475
29	0.045967340	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=104317 Win=60416 Len=0 TSval=1920273475 TSecr=766852818
30	0.045987016	200.0.0.254	200.0.0.1	TCP	26130	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=104317 Ack=29 Win=42496 Len=26064 TSval=766852818 TSecr=1920273475
31	0.046002093	200.0.0.254	200.0.0.1	TCP	23234	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=130381 Ack=29 Win=42496 Len=23168 TSval=766852818 TSecr=1920273475
32	0.046151075	200.0.0.254	200.0.0.1	TCP	2962	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=135453 Ack=29 Win=42496 Len=2996 TSval=766852818 TSecr=1920273475
33	0.046024025	200.0.0.254	200.0.0.1	TCP	8354	200.0.0.254 → 200.0.0.1 [TCP Window Full] Seq=16000 Len=0 MSS=1460 SACK_PERM TSval=766852818 TSecr=1920273475
34	0.046031717	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=150445 Win=60416 Len=0 TSval=1920273475 TSecr=766852818
35	0.046056154	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=164733 Win=60416 Len=0 TSval=1920273475 TSecr=766852818
36	0.046062270	200.0.0.254	200.0.0.1	TCP	18890	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=164733 Ack=29 Win=42496 Len=18824 TSval=766852818 TSecr=1920273475
37	0.046068095	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=193557 Win=60416 Len=0 TSval=1920273475 TSecr=766852818
38	0.046071152	200.0.0.254	200.0.0.1	TCP	466	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=193557 Ack=29 Win=42496 Len=460 TSval=766852818 TSecr=1920273475
39	0.046085989	200.0.0.254	200.0.0.1	TCP	27578	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=193557 Ack=29 Win=42496 Len=27512 TSval=766852818 TSecr=1920273475
40	0.046099316	200.0.0.254	200.0.0.1	TCP	4410	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=211409 Ack=29 Win=42496 Len=4344 TSval=766852818 TSecr=1920273475
41	0.046104608	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=215813 Win=63488 Len=0 TSval=1920273475 TSecr=766852818
42	0.046113927	200.0.0.254	200.0.0.1	TCP	23234	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=215813 Ack=29 Win=42496 Len=23168 TSval=766852818 TSecr=1920273475
43	0.046123585	200.0.0.254	200.0.0.1	TCP	4410	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=238981 Ack=29 Win=42496 Len=4344 TSval=766852818 TSecr=1920273475
44	0.046131444	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=243325 Win=72192 Len=0 TSval=1920273475 TSecr=766852818
45	0.046150155	200.0.0.254	200.0.0.1	TCP	23234	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=243325 Ack=29 Win=42496 Len=23168 TSval=766852818 TSecr=1920273475
46	0.046155605	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=260493 Win=118784 Len=0 TSval=1920273475 TSecr=766852818
47	0.046160698	200.0.0.254	200.0.0.1	TCP	12874	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=260493 Ack=29 Win=42496 Len=12808 TSval=766852818 TSecr=1920273475
48	0.046173877	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=279391 Win=144384 Len=0 TSval=1920273475 TSecr=766852818
49	0.046190658	200.0.0.254	200.0.0.1	TCP	14770	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=279391 Ack=29 Win=42496 Len=14704 TSval=766852818 TSecr=1920273475
50	0.046200446	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=294065 Win=173568 Len=0 TSval=1920273475 TSecr=766852818
51	0.046205122	200.0.0.254	200.0.0.1	TCP	27578	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=294065 Ack=29 Win=42496 Len=27512 TSval=766852818 TSecr=1920273475
52	0.046214087	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=321517 Win=220352 Len=0 TSval=1920273475 TSecr=766852818
53	0.046220403	200.0.0.254	200.0.0.1	TCP	27578	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=321517 Ack=29 Win=42496 Len=27512 TSval=766852818 TSecr=1920273475
54	0.046226178	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=340029 Win=283648 Len=0 TSval=1920273475 TSecr=766852818
55	0.046234474	200.0.0.254	200.0.0.1	TCP	27578	200.0.0.254 → 200.0.0.1 [PSH, ACK] Seq=340029 Ack=29 Win=42496 Len=27512 TSval=766852818 TSecr=1920273475
56	0.046238926	200.0.0.1	200.0.0.254	TCP	66	200.0.0.1 → 200.0.0.254 [ACK] Seq=29 Ack=376541 Win=338432 Len=0 TSval=1920273475 TSecr=766852818

Figura 4: Captura en Wireshark desde la perspectiva del host público h1. Se aprecia el proceso completo de traducción IP/Puerto y el correcto flujo de banderas TCP.

9. Conclusión

En este trabajo práctico se desarrolló una implementación de NAT con traducción de puertos (PAT) utilizando el paradigma de Redes Definidas por Software. La solución fue construida sobre un controlador POX y una topología emulada mediante Mininet, permitiendo integrar conceptos de SDN, OpenFlow y traducción de direcciones en un mismo entorno de trabajo.

Los objetivos planteados por la consigna fueron alcanzados satisfactoriamente. Se logró extender la topología original para soportar múltiples hosts privados, implementar un mecanismo de resolución ARP completamente dinámico, mantener el estado de las conexiones activas mediante tablas NAT y realizar correctamente la traducción de direcciones IP y puertos para tráfico TCP y UDP.

Asimismo, se implementó una estrategia de instalación de flujos que permite que el switch procese de manera autónoma la mayor parte del tráfico una vez establecida cada conexión. Esto reduce significativamente la intervención del controlador y refleja uno de los principios fundamentales de las redes definidas por software: la separación entre el plano de control y el plano de datos.

Las pruebas realizadas demostraron que múltiples hosts privados pueden compartir una única dirección IP pública y mantener conexiones simultáneas sin interferencias entre sí. Además, la utilización de mecanismos de expiración y liberación automática de recursos permitió mantener un uso eficiente de los puertos públicos disponibles y de las estructuras de estado del controlador.

En conclusión, el desarrollo realizado permitió comprender de manera práctica el funcionamiento interno de un NAT/PAT y su implementación sobre una arquitectura SDN. La experiencia también puso de manifiesto la importancia de la gestión de estado, la resolución dinámica de direcciones y la instalación eficiente de reglas de forwarding para construir servicios de red complejos sobre OpenFlow.